



Design and Analysis of Algorithms

Chapter 6 – Greedy Algorithms

Optimization Problems

- ❖ Optimization Problem is a problem which demand or require Minimum or Maximum results.
- ❖ Some problems can have many possible/ feasible solutions with each solution having a specific cost. We wish to find the best solution with the optimal cost.
 - ❖ *Maximization problem* finds a solution with maximum cost
 - ❖ *Minimization problem* finds a solution with minimum cost
- ❖ A set of choices must be made in order to arrive at an optimal (min/max) solution, subject to some constraints.
- ❖ Is “Sorting a sequence of numbers” optimization problem?

The Greedy Method

- ◆ An optimization problem is one in which you want to find, not just *a* solution, but the *best* solution
 - ◆ A “greedy algorithm” works well for most optimization problems
- ◆ Greedy method suggests that one can devise an algorithm that works in phases:
 - ◆ At each phase, take one input at a time (from the ordered input according to the selection criteria) and decide whether it is an optimal solution.

Feasible vs. Optimal Solution

- ◆ Greedy method solves problem by making a sequence of decisions.
 - Decisions are made one by one in some order.
 - Each decision is made using a greedy criterion.
 - A decision, once made, is (usually) not changed later.
- ◆ Given n inputs we are required to obtain a subset that satisfies some constraints
 - ◆ Any subset that satisfies the given constraints is called a **feasible solution**.
 - ◆ A feasible solution that either maximizes or minimizes a given objective function is called an **optimal solution**.

Greedy Algorithm

- ◆ A greedy algorithm works in phases. At each phase:
 - ◆ makes the best choice available right now, without regard for future consequences
 - ◆ hopes to end up at a *global* optimum by choosing a *local* optimum at each step. For some problem, it works
- ◆ Greedy algorithms sometimes works well for optimization problems
- ◆ Greedy algorithms tend to be easier to code
- ◆ Greedy algorithms frequently used in everyday problem solving
 - ◆ Choosing a job
 - ◆ Route finding
 - ◆ Invest on stocks

Greedy Algorithm

- ◆ How to know if a greedy algorithm will solve a particular optimization problem?
- ◆ Two key ingredients
 - ◆ Greedy choice property
 - ◆ Optimal sub-structure
- ◆ If a problem has these properties, then we can develop a greedy algorithm for it.

Greedy Algorithm

- ◆ Greedy choice property: A globally optimal solution can be arrived at by making a locally optimal (greedy) choice
 - ◆ Make whatever choice seems best at the moment and then solve the sub-problem arising after the choice is made
 - ◆ The choice made by a greedy algorithm may depend on choices so far, but it cannot depend on any future choices or on the solutions to sub-problems
- ◆ Optimal sub-structure: A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to sub-problems

Steps for achieving a Greedy Algorithm

- ❖ **Feasible:** Here, we check whether it satisfies all possible constraints or not, to option at least one solution to our problems.
- ❖ **Local Optimal Choice:** In this, the choice should be the optimum which is selected from the currently available.
- ❖ **Unalterable:** Once the decision is made, at any subsequence step that option is not altered.

General Algorithm for Greedy Method

```
procedure greedy (A, n)
    Solution  $\leftarrow \{ \}$ ; // set that will hold the solution set.
    FOR i = 1 to n DO
        x = SELECT (A)
        IF FEASIBLE (Solution, x) THEN
            Solution = UNION (Solution, x)
        end if
    end FOR
    RETURN Solution
end procedure
```

- ◆ **SELECT function**: selects the most promising candidates from $A[]$ and removes it from the list.
- ◆ **FEASIBLE function**: a Boolean valued function that determines whether x can be included into the solution vector.
- ◆ **UNION function**: combines x with the solutions

Knapsack Problem

Knapsack Problem

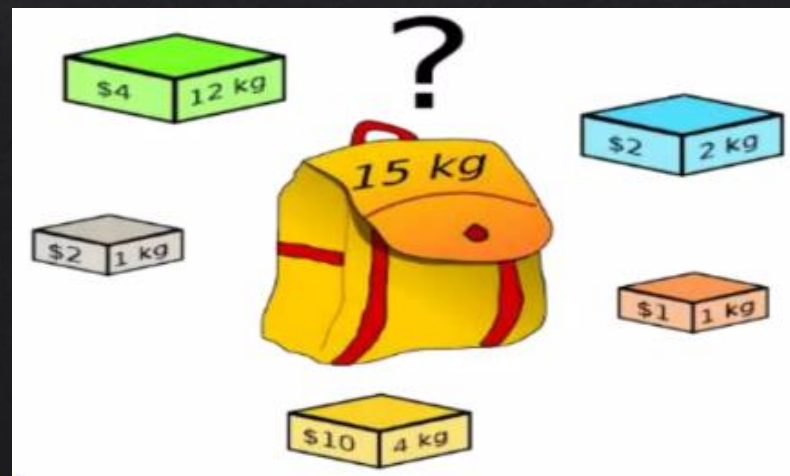
You are given the following:

- ◇ A knapsack (kind of shoulder bag) with limited weight capacity.
- ◇ Few items each having some weight and value.

The problem states:

Which items should be placed into the knapsack such that –

- ◇ The value or profit obtained by putting the items into the knapsack is maximum and
- ◇ The weight limit of the knapsack does not exceed.



Knapsack Problem

◇ Example:

Objects	1	2	3	4	5	6	7
Profit (P)	10	5	15	7	6	18	3
Weight (W)	2	3	5	7	1	4	1

◇ Maximum (M) = 15 Kg

◇ Number Items (N) = 7

Objectives: Max $\sum x_i P_i$

Condition: $\sum x_i W_i \leq M$



Knapsack Problem

Solution:-

Objects	1	2	3	4	5	6	7
Profit (P)	10	5	15	7	6	18	3
Weight (W)	2	3	5	7	1	4	1
Profit/Weight	5	1.6	3	1	6	4.5	3
Selected (X)	1	2/3	1	0	1	1	1



$15 \text{ kg} - 1 \text{ kg} = 14 \text{ kg}$
 $14 \text{ kg} - 2 \text{ kg} = 12 \text{ kg}$
 $12 \text{ kg} - 4 \text{ kg} = 8 \text{ kg}$
 $8 \text{ kg} - 5 \text{ kg} = 3 \text{ kg}$
 $3 \text{ kg} - 1 \text{ kg} = 2 \text{ kg}$
 $2 \text{ kg} - 2 \text{ kg} = 0 \text{ kg}$

◇ $\sum X; W; = 1*2 + 2/3*3 + 1*5 + 0*7 + 1*1 + 1*4 + 1*1 = 15 \text{ Kg}$

◇ $\sum X; P; = 1*10 + 2/3*5 + 1*15 + 0*7 + 1*6 + 1*18 + 1*3 = 55.3 \text{ Cost}$

Huffman Codes

Huffman Codes

- ❖ A Huffman code is a particular type of optimal prefix code that is commonly used for lossless data compression.
- ❖ It is a **compression Technique** used for reducing the size of data or message.
- ❖ The process of finding or using such a code is Huffman coding, an algorithm developed by **David A. Huffman** while he was a student at **MIT**, and published in the **1952** paper "*A Method for the Construction of Minimum-Redundancy Codes*."

Huffman Codes

◇ Example:

Message = BCCABBDDECCBBAEDDCC

Length = 20

ASCII = 8 bit

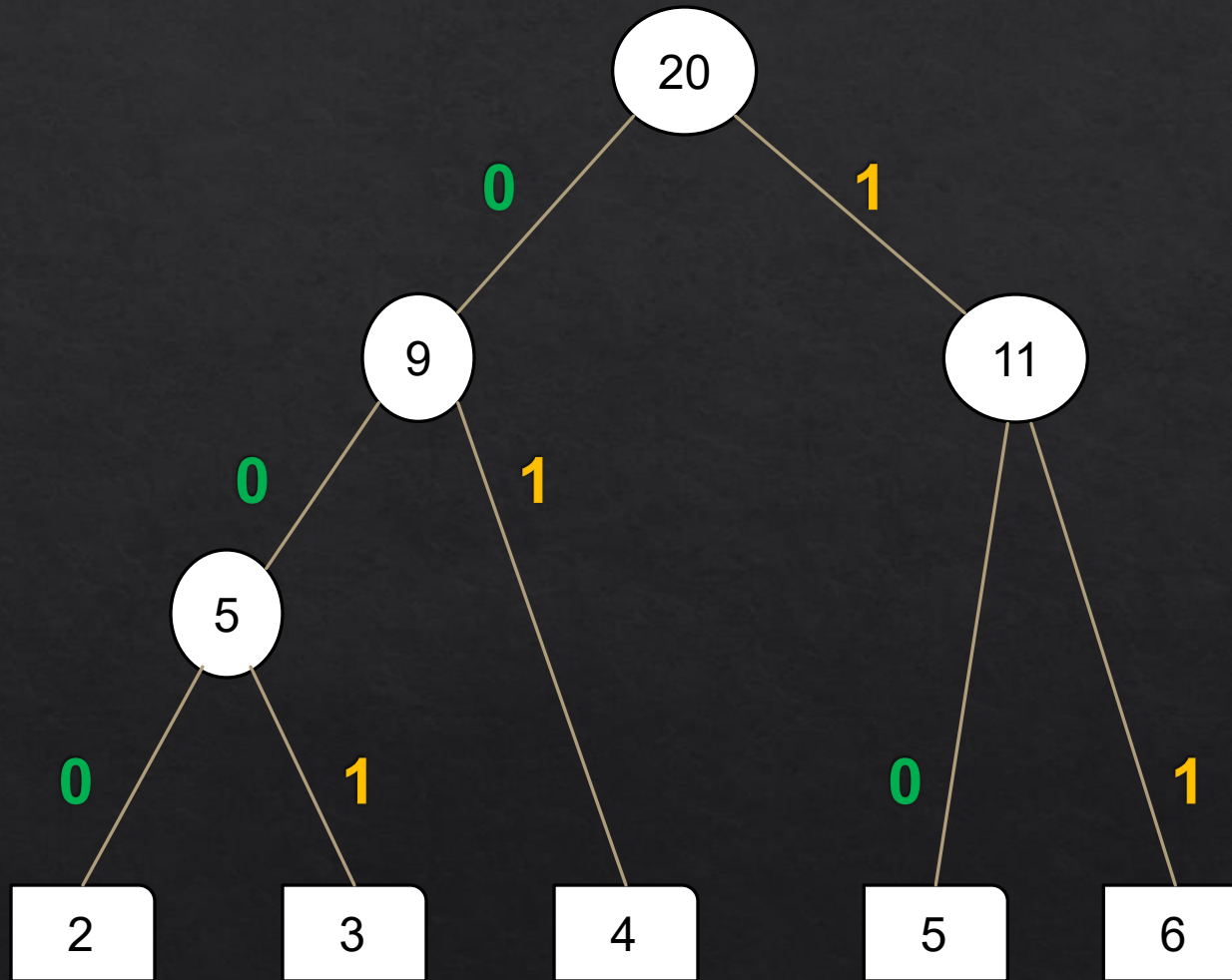
$8 * 20 = 160$ bits

Character	Count	Code	
A	3	001	$3 * 3 = 9$
B	5	10	$5 * 2 = 10$
C	6	11	$6 * 2 = 12$
D	4	01	$4 * 2 = 8$
E	2	000	$2 * 3 = 6$

$5 * 8 = 40$ bits 20 12 bits 45 bits

	A = 65 = 01000001
	B = 66 = 01000010
	C = 67 = 01000011
	D = 68 = 01000100
	E = 69 = 01000101

Cont..



Minimum Spanning Tree

Minimum Spanning Trees

Problem: Laying Telephone Wire



Minimize the total length of wire connecting the customers

Minimum Spanning Tree (MST)

- ◆ Assume you have an undirected graph $G = (V, E)$ with weights assigned to edges.
 - The objective is “use smallest set of edges of the given graph to connect everything together”. How?
 - A minimum spanning tree is a least-cost subset of the edges of a graph that connects all the nodes
- ◆ MST is a sub-graph of an undirected weighted graph G , such that:
 - It is a tree (i.e., it is acyclic)
 - It covers all the vertices V
 - contains $|V| - 1$ edges
 - The total cost associated with tree edges is the minimum among all possible spanning trees
- ◆ **Applications of MST**

Network design, road planning, etc.

How can we generate a MST?

- ◆ A **MST** is a least-cost subset of the edges of a graph that connects all the nodes
- ◆ A greedy method to obtain a minimum-cost spanning tree builds this tree edge by edge.
- ◆ The next edge to include is chosen according to some optimization criterion
- ◆ Criteria: to choose an edge that results in a minimum increase in the sum of the costs of the edges so far included.
- ◆ General procedure
 - Start by picking any node & adding it to the tree
 - Repeatedly: Pick any least-cost edge from a node in the tree to a node not in the tree, & add the edge and new node to the tree
 - Stop when all nodes have been added to the tree
- ◆ Two techniques: **Prim** and **Kruskal** Algorithms

Prim's Algorithm

- ◆ Prim's: always takes the lowest-cost edge between nodes in the spanning tree and nodes not yet in the spanning tree.
- ◆ If A is the set of edges selected so far, then A forms a tree:
- ◆ The next edge (u, v) to be included in A is a minimum cost edge not in A such that $A \cup \{(u, v)\}$ is also a tree, where u is in the tree & v is not.

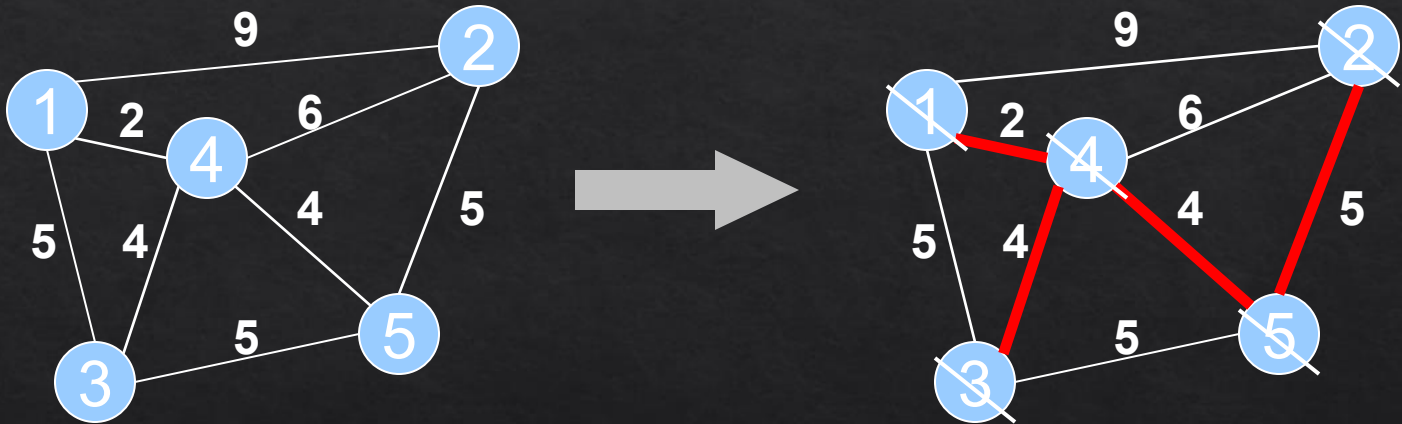
Property:

- ◆ at each step, we add the edge (u, v) such that the weight of (u, v) is minimum among all edges.
- ◆ This spanning tree grows by one new node and edge at each iteration.
- ◆ When all vertices have been included, we have a minimum cost spanning tree for the graph.

Prim's Algorithm

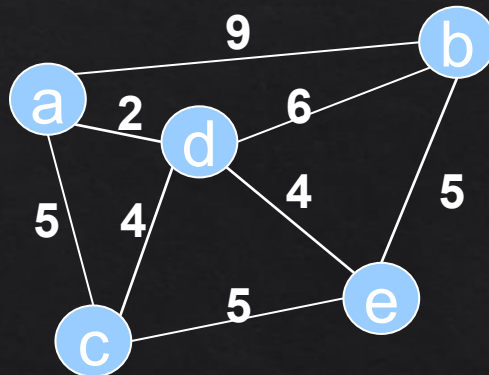
Example:

- Find the minimum spanning tree using Prim algorithm



Kruskal's Algorithm

- ◇ Kruskal algorithm: Always tries the lowest-cost remaining edge
- ◇ It considers the edges of the graph in increasing order of cost.
- ◇ In this approach, the set T of edges so far selected for the spanning tree may not be a tree at all stages in the algorithm. But it is possible to complete T into a tree.
 - Create a forest of trees from the vertices
 - Repeatedly merge trees by adding “**safe edges**” until only one tree remains. A “safe edge” is an edge of minimum weight which does not create a cycle
 - Example:



Initially there is a forest:

$\{a\}, \{b\}, \{c\}, \{d\}, \{e\}$

$E = \{(a, d), (c, d), (d, e), (a, c), (b, e), (c, e), (b, d), (a, b)\}$

THE END

THANK YOU!